

AWS

INFRASTRUCTURE LAB MANUAL

HANDS-ON LABS TO BUILD REAL CLOUD INFRASTRUCTURE
STEP BY STEP



- ✓ REAL AWS SERVICES
- ✓ STEP BY STEP LABS
- ✓ PRACTICAL EXAMPLES
- ✓ TROUBLESHOOTING GUIDES

- ✓ BEST PRACTICES
- ✓ SECURITY AWARENESS
- ✓ PRODUCTION INSIGHTS
- ✓ FINAL HANDS-ON CHALLENGE



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Your First AWS Infrastructure Lab

1. What You Will Learn

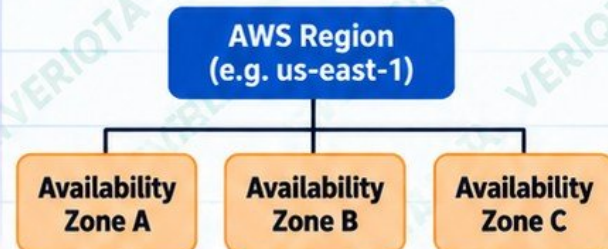
- ✓ Understand the AWS lab environment
- ✓ Explore AWS Regions and Availability Zones
- ✓ Use the AWS Management Console
- ✓ Know what we will build in this lab series
- ✓ Understand basic AWS costs and free tier

2. What is AWS?

AWS (Amazon Web Services) is a cloud platform that provides computing resources over the internet. You can create servers, storage, networks, databases and many other services without buying physical hardware.

3. AWS Global Infrastructure

AWS has multiple Regions around the world. Each Region contains multiple isolated Availability Zones (AZs).



- Regions are separate geographic locations.
- Each Region has multiple Availability Zones.
- AZs are isolated data centers with independent power, networking and cooling.
- Using multiple AZs improves availability.

4. AWS Management Console

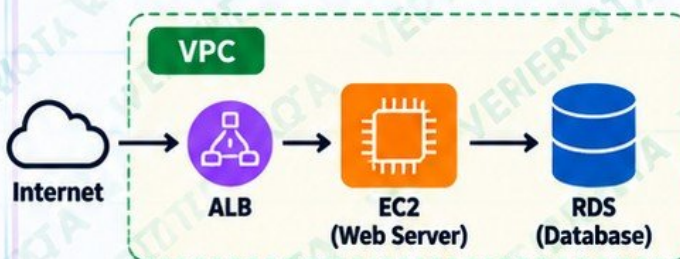
The AWS Management Console is a web interface where you create and manage AWS resources.



- Sign in with your IAM user (not root for daily work).
- Search for services using the search bar.
- Each service has its own dashboard and settings.

5. What We Will Build

In this lab series, we will build a simple and realistic AWS infrastructure environment.



This environment will help you understand how real applications are deployed in AWS.

6. Basic Cost Awareness

- AWS charges based on the resources you use.
- You can use the AWS Free Tier for many services.
- Always stop or delete resources when you are not using them.
- Some resources (like EC2, RDS, and Load Balancers) incur charges if left running.
- Use the AWS Pricing Calculator to estimate costs.



IMPORTANT

- Do not leave resources running after the lab.
- Enable billing alerts.
- Use small instance types (t2.micro or t3.micro) for practice.



Junior Engineer Tip

Start in one Region (for example, us-east-1) and use the Free Tier. Explore, experiment and always clean up resources after each lab.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

Secure Your AWS Account First

1. Root User vs IAM User



Root User

- Created when you open your AWS account
- Has full access to everything
- Use only for account recovery and critical tasks
- Do not use for daily work



IAM User

- Created by you
- Can be given limited permissions
- Used for daily work
- Follow the principle of least privilege
- More secure than root user

2. Enable Multi-Factor Authentication (MFA)

MFA adds an extra layer of security. Even if your password is stolen, an attacker cannot access your account without the second factor.

MFA options:

- Authenticator app (recommended)
- Hardware security key
- SMS (less secure)



3. Why You Should Not Use Root for Daily Work

- ✗ If your credentials are compromised, the entire account is at risk.
- ✗ Root user has unrestricted access.
- ✗ Mistakes can delete important resources.
- ✗ It is difficult to track who made changes.
- ✗ AWS recommends using IAM users for daily tasks.

4. Principle of Least Privilege

- ✓ Give only the permissions a user needs.
- ✓ Avoid giving full administrator access.
- ✓ Use managed policies where possible.
- ✓ Review and remove unused permissions.
- ✓ Use IAM groups to manage permissions.

5. Basic AWS Account Security Checklist

- ✓ Enable MFA for the root user.
- ✓ Create an IAM user for daily work.
- ✓ Use strong and unique passwords.
- ✓ Apply least privilege permissions.
- ✓ Enable AWS IAM Identity Center (for multiple users).
- ✓ Enable billing alerts.
- ✓ Review account activity with CloudTrail.
- ✓ Keep recovery options up to date (email and phone).

6. Example IAM User Setup



Recommended starting permissions for a junior engineer:

- Use AWS managed policies (e.g. AmazonEC2ReadOnly)
- Only add permissions you need for the lab
- Avoid AdministratorAccess



Important Reminders

- Do not share your AWS credentials with anyone.
- Use MFA on all IAM users.
- Regularly review users, groups and permissions.

- Delete unused IAM users.
- Use strong passwords and rotate them.
- Monitor account activity using AWS CloudTrail.
- Set up billing alerts to avoid unexpected charges.



Junior Engineer Tip

Always use an IAM user with MFA for your daily work. Keep the root user secure and use it only when necessary.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Create an IAM User and Role

1. What is IAM?

IAM (Identity and Access Management) controls who can access AWS resources and what they can do.

- Create users and groups
- Manage permissions with policies
- Provide temporary access with roles
- Follow the principle of least privilege

2. Authentication vs Authorization

Authentication (Who are you?)



- Proves your identity.
- Usually done with username and password or access keys.
- Example: IAM user login, MFA.

Authorization (What can you do?)



- Determines your permissions.
- Controlled by IAM policies.
- Example: Allow user to start EC2 but not delete it.

3. Key IAM Components



IAM User

- A person or application that needs access to AWS.
- Has credentials and permissions.



IAM Group

- A collection of users.
- Makes it easy to manage permissions.



IAM Policy

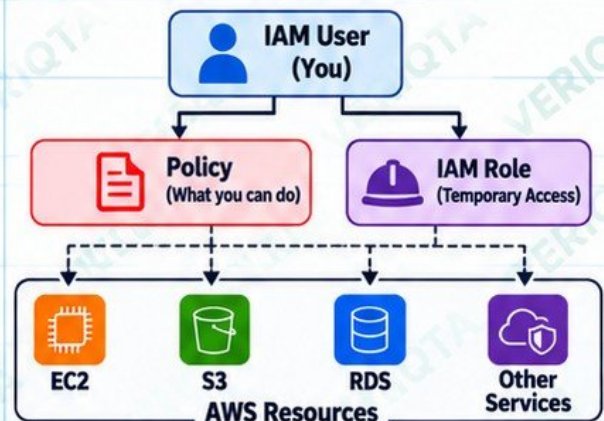
- A JSON document that defines permissions.
- Can be attached to users, groups or roles.



IAM Role

- Provides temporary permissions.
- Used by AWS services or trusted users.
- No long-term credentials.

4. IAM Architecture



5. Hands-On: Create an IAM User

- 1 Sign in to the AWS Management Console as root user.
- 2 Open IAM and click Users → Add user.
- 3 Enter a username (e.g. devops-lab-user).
- 4 Choose AWS Management Console access.
- 5 Set a strong password and require password reset.
- 6 Add the user to a group (or attach a policy directly).
- 7 Enable MFA for the user.

6. Attach a Policy to a User or Group

- 1 Create a policy (use AWS managed or custom).
- 2 Attach the policy to the user or group.
- 3 Example: attach AmazonEC2ReadOnlyAccess for a read-only user.

7. Create an IAM Role (Example for EC2)

- 1 Open IAM → Roles → Create role.
- 2 Choose a trusted entity (e.g. EC2).
- 3 Attach a policy (e.g. AmazonS3ReadOnlyAccess).
- 4 Give the role a name and create it.



Important Security Practices

- Do not use the root user for daily work.
- Enable MFA for all users.
- Use least privilege permissions.
- Review and remove unused users and permissions.
- Rotate access keys if used.



Junior Engineer Tip

- Create individual IAM users for each team member.
- Use groups to manage permissions.
- Use roles instead of long-term access keys whenever possible.
- Start with AWS managed policies, then create custom policies when needed.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Build Your First VPC

1. What is a VPC?

A VPC (Virtual Private Cloud) is your own isolated network in AWS. It allows you to define IP address ranges, subnets, route tables and network settings for your resources.

- You control the network design.
- It is logically isolated from other AWS users.
- It can be connected to the internet or kept private.
- You use a VPC to run secure and organised applications.

2. Why Do Workloads Need Networks?

- ✓ Applications need a network to communicate with each other.
- ✓ You decide how traffic flows (internet, private, or both).
- ✓ You can control security using subnets, route tables and security groups.
- ✓ You can connect the VPC to other networks later (e.g. on-premises or another VPC).
- ✓ It helps you separate environments (dev, test, prod).

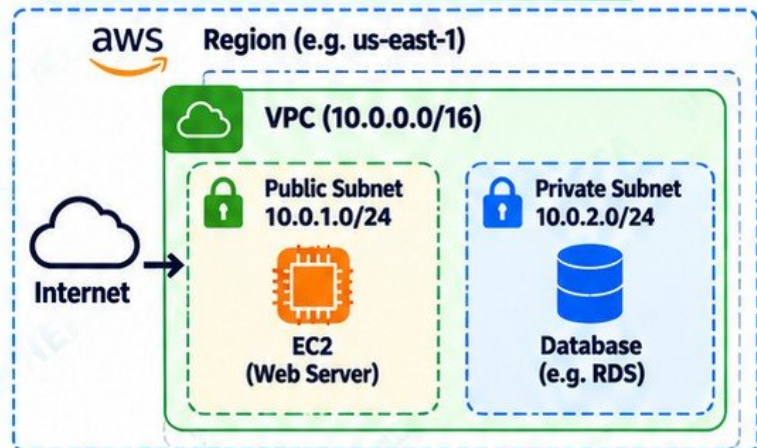
3. Understanding CIDR Notation

CIDR (Classless Inter-Domain Routing) defines the size of your IP address range.

- Example: 10.0.0.0/16
- /16 means 65,536 IP addresses (10.0.0.0 - 10.0.255.255)
- Common VPC ranges: /16 or /24
- Do not overlap with your on-premises network if you plan to connect later.

CIDR	Total IP Addresses	Use Case
/16	65,536	Large environment
/20	4,096	Medium environment
/24	256	Small lab or test

4. VPC Architecture (Basic View)



5. Hands-On: Create a VPC

- 1 Sign in to the AWS Management Console.
- 2 Open the VPC service.
- 3 Click Create VPC.
- 4 Choose VPC only (not the full VPC and more option).
- 5 Enter a name (e.g. devops-lab-vpc).
- 6 Enter a CIDR block (e.g. 10.0.0.0/16).
- 7 Leave other settings as default and click Create VPC.

6. Verify the VPC

- ✓ Open your VPC and confirm it was created.
- ✓ Check the CIDR block.
- ✓ Note the VPC ID (you will use it in later labs).
- ✓ You can also view the default subnets route tables and network settings.

aws VPCs			
	Name	VPC ID	IPv4 CIDR block
☐	devops-lab-vpc	vpc-0a1b2c3d4e5f6g7h	10.0.0.0/16

7. Choosing an Address Range (Best Practice)

- ✓ Use a private IP range (e.g. 10.0.0.0/16 or 192.168.0.0/16).
- ✓ Choose a range that is large enough for your environment.
- ✓ Avoid overlapping with existing networks.
- ✓ You can create multiple subnets from this range later.

8. Important Notes

- ! You can create only one VPC per region with the same CIDR.
- ! Plan your IP address range before creating the VPC.
- ! You cannot change the CIDR block after creation.
- ! A VPC by itself has no internet access until you add an Internet Gateway and configure routing (next lab).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Create Public and Private Subnets

1. What is a Subnet?

A subnet is a smaller range of IP addresses within your VPC. You split your VPC into subnets to organize resources, control traffic and place workloads in different Availability Zones (AZs).

- Each subnet has its own CIDR range.
- Subnets can be public or private.
- You choose which AZ each subnet belongs to.

2. Public vs Private Subnet



Public Subnet

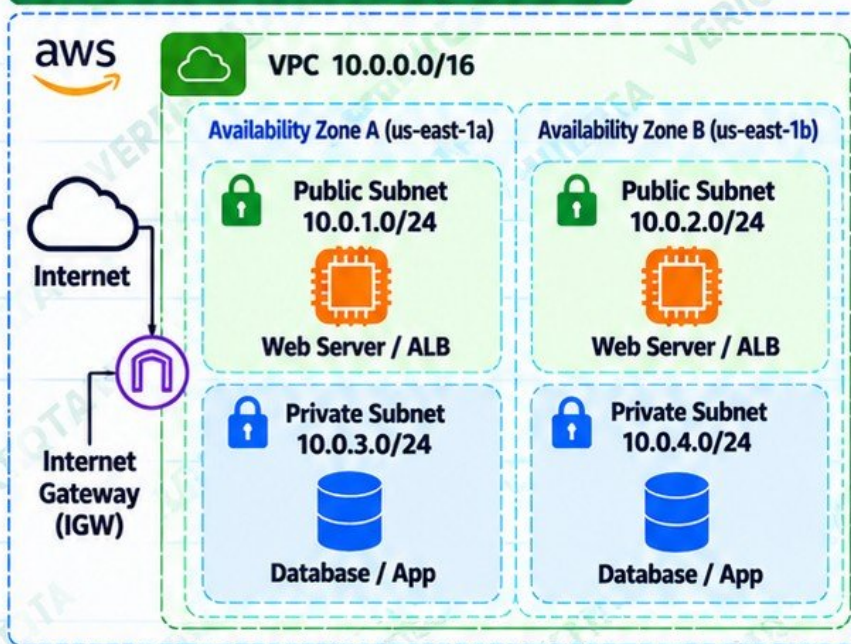
- Can route traffic to the internet.
- Usually has a route to an Internet Gateway (IGW).
- Used for resources that need internet access (e.g. web servers, load balancers).



Private Subnet

- No direct access to the internet.
- No route to the Internet Gateway.
- Used for internal resources (e.g. databases, backend services).
- Can access the internet using a NAT Gateway (if needed).

3. Example Subnet Design (2 AZs)



4. CIDR Planning Example

Subnet	CIDR Block	Purpose
VPC	10.0.0.0/16	Entire network
Public Subnet A	10.0.1.0/24	Web/ALB (AZ A)
Public Subnet B	10.0.2.0/24	Web/ALB (AZ B)
Private Subnet A	10.0.3.0/24	App/Database (AZ A)
Private Subnet B	10.0.4.0/24	App/Database (AZ B)

5. Hands-On: Create Subnets

- Open the VPC you created in the previous lab.
- Go to Subnets → Create subnet.
- Select the VPC and choose an Availability Zone.
- Enter a subnet name and CIDR block.
- Create one public and one private subnet in each AZ.
- Repeat for the second AZ.

6. Verify the Subnets

- Go to the Subnets page and confirm they were created.
- Check the CIDR blocks and Availability Zones.
- Ensure there is no overlapping CIDR range.
- Confirm the public subnets are in different AZs.
- Check the route table associated with each subnet.

7. Important Notes

- A subnet is not public just because it has a public IP.
- A public subnet must have a route to an Internet Gateway.
- Keep subnets in different AZs for high availability.
- Plan your IP ranges to avoid overlap with on-premises networks.
- Use private subnets for databases and internal services.



Junior Engineer Tip

Use at least two AZs for important workloads, even in your labs. This helps you understand high availability from the start.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



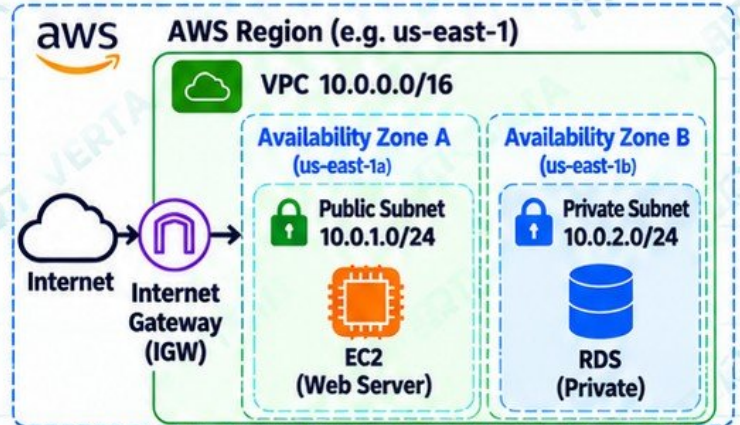
VERIQTA

Connect the VPC to the Internet

1. What is an Internet Gateway (IGW)?

- An Internet Gateway (IGW) is a horizontally scaled, highly available component that allows communication between your VPC and the internet.
- It enables resources in public subnets to send and receive internet traffic.
- You must create an IGW and attach it to your VPC.
- A VPC can have only one IGW attached at a time.

2. Architecture Overview



3. Steps: Create and Attach an IGW

- 1 Open the AWS Management Console.
- 2 Go to VPC → Internet Gateways → Create internet gateway.
- 3 Enter a name (e.g. devops-lab-igw) and create it.
- 4 Select the IGW → Actions → Attach to VPC.
- 5 Choose your VPC (e.g. 10.0.0.0/16) and attach.
- 6 Verify the IGW is attached to your VPC.

4. Route Table Configuration

- A public subnet must have a route to the internet via the Internet Gateway.
- Add a route 0.0.0.0/0 → IGW in the public subnet route table.
- Private subnets do not have a direct route to the internet.
- Route tables control where traffic is sent.

5. Route Table Examples

Public Subnet Route Table	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxx

Private Subnet Route Table	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	(none)

6. Why a Subnet is Not Public Just Because It Has a Public IP

- ✗ A public IP alone does not make a subnet public.
- ✗ The subnet must have a route to an Internet Gateway in its route table.
- ✗ If there is no 0.0.0.0/0 route to an IGW, the instance cannot reach the internet even with a public IP.
- ✗ Network placement is controlled by the subnet's route table, not the IP address.

7. Verify Internet Access

- 1 Launch an EC2 instance in the public subnet.
- 2 Ensure it has a public IP address.
- 3 Connect to the instance and test internet access.
- 4 Run: `curl -I https://aws.amazon.com`
- 5 Check the route table and security group rules.

```
$ curl -I https://aws.amazon.com
HTTP/2 200
content-type: text/html
content-length: 12345
server: AmazonS3
...
```

Important Notes

- Create only one IGW per VPC.
- Public subnets must have a route to the IGW.
- Private subnets should not have a direct route to the internet.
- Use Network ACLs and Security Groups for additional control.
- For private subnets that need internet access (e.g. for updates), use a NAT Gateway (next lab).



Junior Engineer Tip

Always check the route table when troubleshooting internet connectivity.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

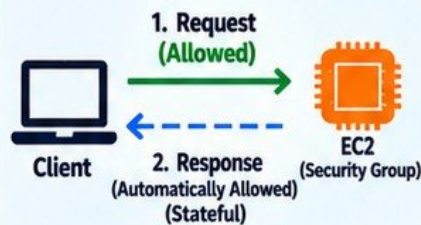


Control Traffic with Security Groups

1. What is a Security Group?

- A security group is a virtual firewall that controls inbound and outbound traffic to your AWS resources (e.g. EC2, RDS, ALB).
- It allows only the traffic you explicitly permit.
- It is stateful, which means if you allow an incoming request, the response traffic is automatically allowed.

2. Stateful Filtering



- You do not need to create a rule for the response traffic.
- If inbound traffic is allowed, return traffic is automatically allowed.

3. Inbound vs Outbound



Inbound Rules
(Traffic coming to your resource)

- Control who can access your resource.
- Specify source (IP, security group, or prefix list).
- Specify protocol and port.



Outbound Rules
(Traffic leaving your resource)

- Control where your resource can go.
- Specify destination (IP, security group or prefix list).
- Specify protocol and port.
- By default, all outbound traffic is allowed.

4. Security Group Components



Rule Type

- Inbound or Outbound



Protocol

- TCP
- UDP
- ICMP
- All traffic



Port Range

- Single port (e.g. 22)
- Range (e.g. 80–90)
- All ports



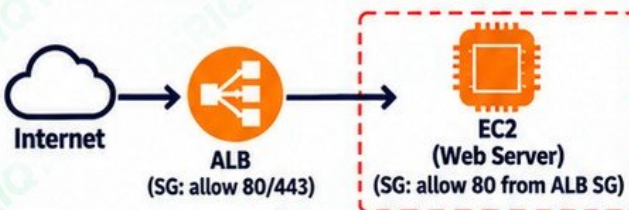
Source / Destination

- IP address (e.g. 203.0.113.5)
- CIDR range (e.g. 10.0.0.0/16)
- Security group
- Prefix list

5. Common Inbound Rule Examples

Purpose	Protocol	Port	Source
SSH (Linux)	TCP	22	Your IP (e.g. 203.0.113.5/32)
HTTP (Web)	TCP	80	0.0.0.0/0 (use only for public web)
HTTPS (Web)	TCP	443	0.0.0.0/0 (use only for public web)
RDP (Windows)	TCP	3389	Your IP (e.g. 203.0.113.5/32)
MySQL (Database)	TCP	3306	App SG or VPC CIDR (e.g. 10.0.0.0/16)

6. Example: Web Server Security Group



Inbound Rules for EC2

Type	Protocol	Port	Source
HTTP	TCP	80	ALB Security Group
SSH	TCP	22	Your IP (optional)

Outbound Rules for EC2

Type	Protocol	Port	Destination
All Traffic	All	All	0.0.0.0/0 (default – can be restricted)

7. Avoid 0.0.0.0/0 Unnecessarily

- ✗ Do not use 0.0.0.0/0 for SSH, RDP or databases.
- ✗ It allows anyone on the internet to access your resource.
- ✗ Use your IP, a specific CIDR range or a security group instead.
- ✗ Only use 0.0.0.0/0 for public web services (HTTP/HTTPS).
- ✗ Review and remove unused rules regularly.

8. Hands-On: Create a Security Group

1. Open the EC2 console → Security Groups → Create security group.
2. Enter a name and description (e.g. web-sg).
3. Add inbound rules (e.g. HTTP from 0.0.0.0/0 and SSH from your IP).
4. Add or keep the default outbound rule (or restrict it).
5. Attach the security group to your EC2 instance or ALB.
6. Test the access and verify it works.



Junior Engineer Tip

Use security groups to follow the principle of least privilege. Allow only the traffic you need and restrict everything else.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

Launch an EC2 Instance

1. What is EC2?

- Amazon EC2 (Elastic Compute Cloud) provides virtual servers (instances) in the cloud.
- You choose the operating system (AMI), instance type, networking and storage.
- It gives you full control, similar to a physical server, but you only pay for what you use.
- Whether it is for web servers, applications, DevOps tools or testing, EC2 is one of the most used AWS services.

2. What AWS Creates Behind the Scenes

				
EC2 Instance (Compute)	EBS Volume (Storage)	Network Interface (ENI)	Security Group (Firewall)	IP Address (Public/Private)
Virtual server in your subnet	Root volume attached to the instance	Primary network interface with a private IP (and public IP if enabled)	Controls inbound and outbound traffic	Private IP (always) Public IP (if enabled)

3. Choose an AMI (Operating System)

- An AMI (Amazon Machine Image) is a template that contains an operating system and software.
- Common options: Amazon Linux, Ubuntu, Red Hat, Windows Server.
- Choose an AMI based on your use case.

Amazon Machine Image (AMI)

Quick Start

My AMIs



Amazon Linux



Ubuntu



Red Hat



Windows



Browse more AMIs
Including AWS, Marketplace and community AMIs

4. Choose an Instance Type

- Instance type determines the CPU, memory and network performance.
- Examples:

Instance Type	vCPU	Memory	Use Case
t3.micro	2	1 GB	Free tier, testing, small projects
t3.small	2	2 GB	Small applications
t3.medium	2	4 GB	Web servers, dev/test
t3.large	2	8 GB	Production workloads
m5.large	2	8 GB	General purpose apps
c5.large	2	4 GB	Compute intensive apps

5. Configure Networking

- Select your VPC and subnet (public or private).
- Assign a public IP if you need internet access (e.g. for web server or SSH).
- Choose the appropriate security group.
- Place the instance in the correct Availability Zone.

6. Configure Storage

- By default, a root EBS volume is created.
- Choose the volume size and type (e.g. gp3).
- You can add additional volumes later if needed.
- EBS volumes persist even if you stop the instance.

7. Hands-On: Launch an EC2 Instance

- Open the EC2 console.
- Click Launch instance.
- Choose an AMI (e.g. Amazon Linux 2023).
- Select an instance type (e.g. t3.micro).
- Configure networking (VPC, subnet, public IP).
- Select or create a security group.
- Configure storage (leave default or adjust).
- Review and click Launch instance.

8. Example Configuration

Setting	Value (Example)
Name	web-server-1
AMI	Amazon Linux 2023
Instance Type	t3.micro
VPC	devops-lab-vpc (10.0.0.0/16)
Subnet	Public Subnet (10.0.1.0/24)
Security Group	web-sg (allow HTTP, SSH)
Public IP	Enabled
Storage	8 GB gp3 (default)
Key Pair	devops-key

9. Verify the Instance

- Go to EC2 → Instances and check the instance state is running.
- Confirm the public and private IP addresses.
- Check the instance type, AZ and other details.
- Verify the security group and subnet.
- Connect to the instance in the next lab (e.g. using SSH).



Junior Engineer Tip

Give your instance a clear name and tag it (e.g. Environment: dev, Project: lab). This makes it easier to identify and manage resources later.

10. Important Notes

- Choose the right instance type to avoid unnecessary costs.
- Always use a key pair for secure access (Linux).
- Keep security group rules restricted (avoid 0.0.0.0/0).
- Use a public subnet and public IP only when internet access is required.
- You can stop the instance to reduce costs, but EBS storage will still incur charges.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

Connect Securely to EC2

1. Why Secure Access Matters?

- You access EC2 using SSH (Linux) with an SSH key pair.
- It is more secure than using a password.
- You should use secure access methods and avoid exposing your instance to unnecessary risk.
- Good access control helps protect your data and workloads.

2. SSH Key Pair



Private Key (Keep Secret)

- Stored on your local machine.
- Used to connect (SSH).
- Never share this file.
- Keep it safe.



Public Key (Stored in AWS)

- Uploaded to AWS when you create the key pair.
- Used by EC2 to verify your identity.
- Cannot be used to connect.

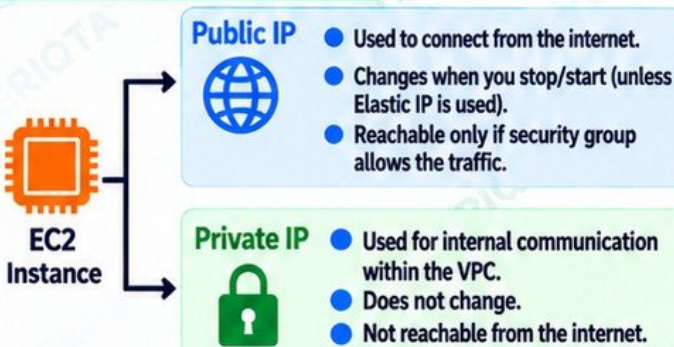
3. Key File Permissions

- Your private key file must have correct permissions.
- On Linux or macOS:

```
$ chmod 400 my-key.pem
```

- If the permissions are too open, SSH will refuse to use the key.

4. Public vs Private IP



5. Connect to Your EC2 Instance



Linux / macOS

```
$ ssh -i my-key.pem ec2-user@<public-ip>
```

- Replace my-key.pem with your key file.
- Use the correct username (e.g. ec2-user for Amazon Linux, ubuntu for Ubuntu).
- Make sure the security group allows SSH (port 22).



Windows (using PowerShell)

```
PS> ssh -i my-key.pem ec2-user@<public-ip>
```

- Use the same command in PowerShell or Windows Terminal.
- Ensure the key file is in the correct path.
- You can also use tools like PuTTY if required.

6. Example: Successful Connection

```
$ ssh -i my-key.pem ec2-user@3.123.45.67
```

```
The authenticity of host '3.123.45.67' can't be established.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '3.123.45.67' (ED25519) to the list of known hosts.
[ec2-user@ip-10-0-1-10 ~]$
```

8. Safer Administrative Access

- Use your own IP address in the security group instead of 0.0.0.0/0.
- Consider using a bastion host or AWS Systems Manager Session Manager (SSM) instead of direct internet access.
- Rotate key pairs if needed.
- Remove access when it is no longer required.
- Use IAM and proper logging for administrative access.

7. Common Connection Issues

Issue	Possible Cause	How to Fix
Connection timed out	Security group or network issue	Check SG rules, public IP, route table, and IGW
Permission denied (publickey)	Incorrect key file or permissions	Use correct key and run <code>chmod 400 my-key.pem</code>
Host key verification failed	Previous key changed	Remove old host key from <code>known_hosts</code> or use a new IP
Connection refused	SSH not allowed	Check security group allows port 22 from your IP
No route to host	Instance has no public IP	Assign a public IP or use bastion/SSM

9. Important Notes

- Never share your private key file.
- Do not use 0.0.0.0/0 for SSH in production.
- A public IP is required for direct SSH from the internet.
- You can use a bastion host or SSM for more secure access.
- If you stop your instance, the public IP may change (unless you use an Elastic IP).



Junior Engineer Tip

Always test SSH access immediately after launch. If you cannot connect, check the security group, public IP and key file permissions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



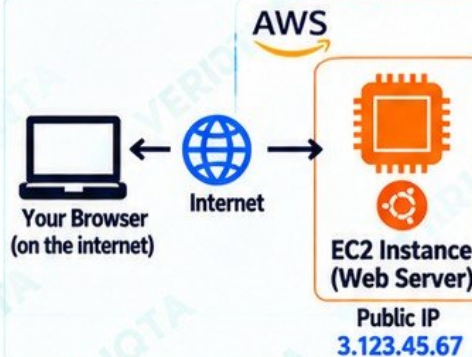
VERIQTA

Deploy a Web Server on EC2

1. What You Will Do

- Install a web server (Apache or Nginx) on an EC2 instance.
- Start the service and verify it locally.
- Allow HTTP traffic using a security group.
- Test the application from a browser.
- Understand how the components work together.

2. Architecture Overview



3. Launch an EC2 Instance

- 1 Open the EC2 console.
- 2 Choose an AMI (e.g. Amazon Linux 2023 or Ubuntu).
- 3 Select an instance type (e.g. t3.micro).
- 4 Choose your VPC, subnet and key pair.
- 5 Create or select a security group (we will allow HTTP in step 5).
- 6 Launch the instance.

4. Install a Web Server

Amazon Linux 2023 (Apache)

```
# Update packages
sudo dnf update -y

# Install Apache
sudo dnf install -y httpd

# Start and enable
sudo systemctl start httpd
sudo systemctl enable httpd
```

Ubuntu (Nginx)

```
# Update packages
sudo apt update -y

# Install Nginx
sudo apt install -y nginx

# Start and enable
sudo systemctl start nginx
sudo systemctl enable nginx
```

5. Verify the Service Locally

Apache (Amazon Linux)

```
# Check service status
sudo systemctl status httpd

# Verify from localhost
curl http://localhost
```

Nginx (Ubuntu)

```
# Check service status
sudo systemctl status nginx

# Verify from localhost
curl http://localhost
```



If the service is running you should see the default web page content in the output.

6. Configure Security Group for HTTP

Type	Protocol	Port	Source	Description
HTTP	TCP	80	0.0.0.0/0 (or your IP)	Allow web traffic
SSH	TCP	22	Your IP (e.g. 203.0.113.5/32)	Allow SSH access

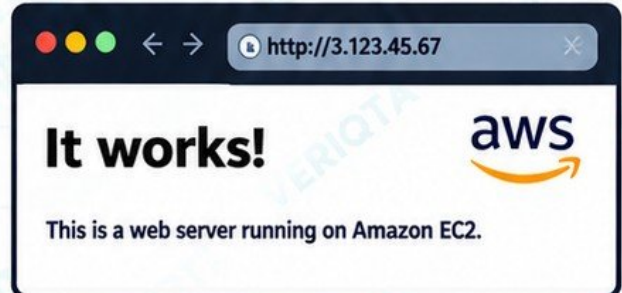


Only open the ports you need. For production, restrict the source instead of using 0.0.0.0/0.

8. Common Issues and Fixes

Issue	Possible Cause	How to Fix
Cannot access the web page	HTTP not allowed in security group	Add inbound rule for port 80
Connection timeout	Instance has no public IP or wrong route	Ensure instance has a public IP and is in a public subnet
Blank or default page not showing	Web server not running	Check service status and start it
Permission denied	Wrong key file or permissions	Use the correct key and run <code>chmod 400 your-key.pem</code>

7. Test from a Browser



9. Important Notes

- ! Make sure your instance has a public IP or an Elastic IP.
- ! HTTP uses port 80 (TCP).
- ! Keep SSH restricted to your IP.
- ! Do not expose unnecessary ports.
- ! Use a custom web page for real applications.
- ! This setup is for learning and testing purposes.



Junior Engineer Tip

After deploying your web server, try replacing the default page with a custom HTML page to understand how web content is served from the instance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



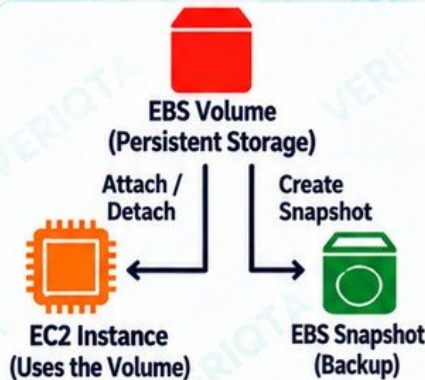
VERIQTA

Attach and Manage EBS Storage

1. What is Amazon EBS?

- Amazon EBS (Elastic Block Store) provides persistent block storage for EC2 instances.
- You can attach, detach, resize and take snapshots.
- Data on EBS persists even if the instance stops (but not if you terminate the volume).
- Common use cases: databases, application data, logs and additional storage.

2. Architecture Overview



3. Create an EBS Volume

- Open the AWS Management Console.
- Go to EC2 → Volumes → Create Volume.
- Choose the size, volume type (e.g. gp3), and Availability Zone.
- Add tags (optional).
- Create the volume.

Example (AWS CLI)

```
$ aws ec2 create-volume
--size 10
--volume-type gp3
--availability-zone us-east-1a
--tag-specifications
"ResourceType=volume,Tags=[(Key=Name,Value=devops-data)]"
```

4. Attach the Volume to an EC2 Instance

- Go to EC2 → Volumes → Select the volume.
- Click Actions → Attach Volume.
- Choose your EC2 instance and device name (e.g. /dev/xvdf).
- Attach the volume.

Example (AWS CLI)

```
$ aws ec2 attach-volume
--volume-id vol-0123456789abcdef0
--instance-id i-0123456789abcdef0
--device /dev/xvdf
```

5. Inspect Block Devices

SSH into your instance and check the new volume.

```
$ lsblk
$ sudo fdisk -l
$ sudo file -s /dev/xvdf
$ sudo blkid
```

Common Device Names	Description
/dev/xvdf	New EBS volume
/dev/nvme1n1	Newer instances (NVMe naming)
/dev/xvda	Root volume (existing)

6. Create a Filesystem

Create a filesystem on the new volume (e.g. ext4).

```
$ sudo mkfs.ext4 /dev/xvdf
```



You only need to create a filesystem once on a new volume.

7. Mount the Volume

- Create a mount point directory.
- Mount the volume.
- Verify the mount.

```
$ sudo mkdir -p /data
$ sudo mount /dev/xvdf /data
$ df -h
```

8. Persist the Mount

Update /etc/fstab so the volume mounts automatically after reboot.

- Get the UUID of the volume.

```
$ sudo blkid /dev/xvdf
```

- Add an entry to /etc/fstab.

```
UUID=<your-uuid> /data ext4 defaults 0 2
```

- Test the configuration.

```
$ sudo mount -a
```

9. Working with Snapshots

- Snapshots are point-in-time backups of your EBS volume.
- They are stored in Amazon S3 and can be used to create new volumes.
- Useful for backup, disaster recovery and migration.

Create a Snapshot (CLI)

```
$ aws ec2 create-snapshot
--volume-id vol-0123456789abcdef0
--description "Backup of devops data"
```

List Snapshots (CLI)

```
$ aws ec2 describe-snapshots
--filters "Name=volume-id,Values=vol-0123456789abcdef0"
```

10. Common Use Cases

- ✓ Store application data (web apps, databases).
- ✓ Store logs and backups.
- ✓ Increase storage for an existing instance.
- ✓ Migrate data using snapshots.
- ✓ Create new volumes from snapshots.

11. Important Notes

- ! EBS volumes persist even if the instance stops.
- ! Data is deleted if you delete the volume.
- ! Choose the right volume type (gp3 is good for most use cases).
- ! Take regular snapshots.
- ! You can attach a volume to only one instance at a time (Multi-Attach is available for specific use cases).



Junior Engineer Tip

Use EBS snapshots regularly to protect important data and test volume recovery.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

Store Objects with Amazon S3

1. What is Amazon S3?

- Amazon S3 (Simple Storage Service) is an object storage service.
- It stores data as objects in buckets.
- It is highly durable, scalable and cost effective.
- You can store any type of file (photos, logs, backups, videos, etc).
- It is not a file system and does not use directories like a traditional disk.

2. How S3 Works



Each object has a unique key (name) and metadata.

3. Create an S3 Bucket

- Open the AWS Management Console.
- Go to S3 → Create bucket.
- Enter a unique bucket name (e.g. veriqa-demo-bucket).
- Choose the AWS Region.
- Keep Block Public Access enabled.
- Create the bucket.

Example (AWS CLI)



```
$ aws s3 mb s3://veriqa-demo-bucket
--region us-east-1
```

4. Upload and Manage Objects

- Upload files to your bucket.
- View, download or delete objects.
- Organize using key names (e.g. images/photo.jpg).

Example (AWS CLI)



```
# Upload a file
$ aws s3 cp myfile.txt
s3://veriqa-demo-bucket/

# List objects
$ aws s3 ls s3://veriqa-demo-bucket/

# Download a file
$ aws s3 cp s3://veriqa-demo-bucket/
myfile.txt .
```

5. Block Public Access



- Prevent accidental public access.
- Keep Block Public Access enabled for new buckets.
- Use bucket policies carefully when you need controlled access.
- Avoid making objects public unless absolutely required.

7. Encryption



- Protects your data at rest.
- Use server-side encryption (SSE-S3 or SSE-KMS).
- You can also use client-side encryption if needed.

6. Versioning



- Keeps multiple versions of an object.
- Helps with recovery from accidental delete or overwrite.
- Useful for backups and important data.
- Can be enabled at the bucket level.

Example (Enable Versioning)



```
$ aws s3api put-bucket-versioning
--bucket veriqa-demo-bucket
--versioning-configuration Status=Enabled
```

Example (Enable SSE-KMS)



```
$ aws s3api put-bucket-encryption
--bucket veriqa-demo-bucket
--server-side-encryption-configuration
'{"Rules":[{"ApplyServerSideEncryptionByDefault":
{"SSEAlgorithm":"aws:kms"}}}]'
```

8. Common S3 Use Cases



9. Important Notes

- Keep Block Public Access enabled.
- Use versioning for important data.
- Enable encryption.
- Use lifecycle policies to manage costs (e.g. move old data to cheaper storage).
- S3 is for object storage, not for running applications or databases.
- Choose the right storage class based on your use case.

10. S3 Storage Classes (Examples)

Storage Class	Use Case
S3 Standard	Frequently accessed data
S3 Intelligent-Tiering	Data with changing access patterns
S3 Standard-IA	Infrequently accessed data
S3 Glacier Instant Retrieval	Long-term backups (quick access)
S3 Glacier Deep Archive	Very long-term archive (lowest cost)



Junior Engineer Tip

Use meaningful bucket names, enable versioning and encryption, and always follow the principle of least privilege for access.



@veriqa



@veriqa



@veriqa



@veriqa



@veriqa



@veriqa



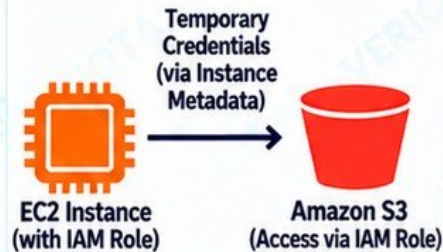
VERIQTA

Give EC2 Access to S3 Safely

1. Why Use an IAM Role Instead of Access Keys?

- IAM roles provide temporary credentials automatically.
- No need to store or manage access keys on the instance.
- More secure and follow AWS best practices.
- Access is automatically rotated and short lived.
- Reduces the risk of credential leakage.

2. Architecture Overview



- ✓ The EC2 instance assumes an IAM role and uses temporary credentials to access S3.

3. Create an IAM Role with S3 Access

- 1 Open the IAM console.
- 2 Create a new role.
- 3 Choose Trusted entity: AWS service and select EC2.
- 4 Attach an S3 policy (e.g. AmazonS3ReadOnlyAccess or a custom policy with least privilege).
- 5 Give the role a name (e.g. ec2-s3-role).
- 6 Create the role.

- ! Use least privilege and allow only the required S3 permissions.

4. Attach the IAM Role to EC2

- 1 Go to the EC2 console.
- 2 Select your instance.
- 3 Choose Actions → Security → Modify IAM role.
- 4 Attach the IAM role (e.g. ec2-s3-role).



The instance will now automatically receive temporary credentials.

5. Verify the IAM Role on EC2

On the EC2 instance, check the IAM role information:

```

$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ec2-s3-role
  
```

6. Use AWS CLI to Access S3

List objects in your S3 bucket:

```
$ aws s3 ls s3://veriqta-demo-bucket/
```

Upload a file to S3:

```
$ aws s3 cp myfile.txt s3://veriqta-demo-bucket/
```

Download a file from S3:

```
$ aws s3 cp s3://veriqta-demo-bucket/myfile.txt .
```

7. Example IAM Policy (Least Privilege)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::veriqta-demo-bucket"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::veriqta-demo-bucket/*"
    }
  ]
}
  
```

8. Temporary Credentials vs Hardcoded Access Keys



IAM Role (Temporary Credentials)

- ✓ Automatically provided by AWS
- ✓ Short lived and rotated
- ✓ No keys stored on the instance
- ✓ More secure
- ✓ Follows AWS best practices
- ✓ Reduced risk of credential leakage



Hardcoded Access Keys

- ✗ Long lived credentials
- ✗ Must be stored on the instance
- ✗ Risk of accidental exposure (e.g. in code or logs)
- ✗ Need to be manually rotated
- ✗ Higher risk if the instance is compromised
- ! Not recommended for EC2 instances

9. Test End to End

- ✓ Attach the IAM role to EC2.
- ✓ Use AWS CLI to list, upload and download files.
- ✓ Verify access works without using access keys.
- ✓ Confirm the instance can only access the allowed S3 resources.

10. Important Notes

- ! Use least privilege permissions.
- ! Do not store access keys on EC2.
- ! Temporary credentials are automatically managed by AWS.
- ! Use IAM roles for all AWS services running on EC2.
- ! Review and monitor access using CloudTrail.



Junior Engineer Tip

Always use IAM roles for EC2 to access AWS services. It is more secure, easier to manage and follows AWS best practices.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

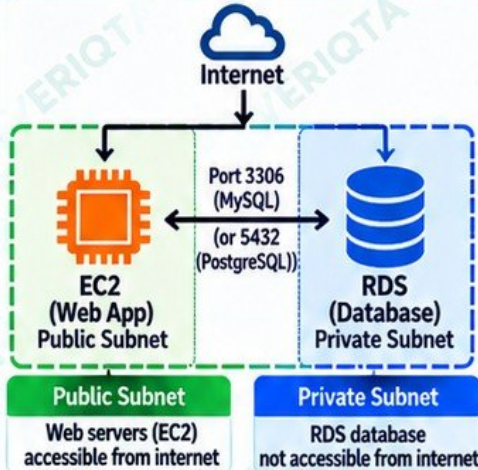


Add a Private Database with RDS

1. What is Amazon RDS?

- Amazon RDS (Relational Database Service) is a managed database service.
- Supports multiple engines (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server).
- Automates backups, patching, monitoring and high availability.
- Deploy the database in private subnets to keep it out of the public internet.
- Access is controlled using security groups.

2. Architecture Overview



3. Deploy an RDS Instance

- 1 Open the RDS console.
- 2 Choose a database engine (e.g. MySQL or PostgreSQL).
- 3 Select instance class and storage.
- 4 Choose a VPC and private subnets.
- 5 Configure database credentials.
- 6 Ensure Public access is set to No.
- 7 Create the database.



Place the RDS instance in private subnets across multiple Availability Zones for high availability.

4. Security Group Configuration

EC2 Security Group (Web Tier)

Type	Protocol	Port	Source	Description
HTTP	TCP	80	0.0.0.0/0	Allow web traffic
SSH	TCP	22	Your IP	Allow SSH access
Custom	TCP	3306	RDS SG	Allow DB access to RDS (MySQL) (or 5432 for PostgreSQL)



Allow database access only from the EC2 security group, not from the internet.

RDS Security Group (Database Tier)

Type	Protocol	Port	Source	Description
MySQL (or PostgreSQL)	TCP	3306 (5432)	EC2 SG	Allow access only from the EC2 security group



Do not use 0.0.0.0/0 for the database. Keep it private.

5. Connect from the Application Tier

From your EC2 instance, connect to the RDS database using the private endpoint.

MySQL Example

```
$ mysql -h veriqa-db.abcdefghijkl.us-east-1.rds.amazonaws.com -u admin -p
```

PostgreSQL Example

```
$ psql -h veriqa-db.abcdefghijkl.us-east-1.rds.amazonaws.com -U admin -d mydb
```

6. Verify the Database is Private

- ✓ The RDS instance has no public IP address.
- ✓ You cannot access it from the internet.
- ✓ It is placed in private subnets.
- ✓ Only the application tier (EC2) can connect.

Check RDS Endpoint (AWS CLI)

```
$ aws rds describe-db-instances \
  --db-instance-identifier veriqa-db \
  --query "DBInstances[0].Endpoint.Address" \
  --output text
veriqa-db.abcdefghijkl.us-east-1.rds.amazonaws.com
```

7. Important Notes

- ! Keep the RDS instance in private subnets.
- ! Do not enable public access.
- ! Use security groups to control access.
- ! Use strong database credentials and consider AWS Secrets Manager for production.
- ! Enable automated backups.
- ! Choose Multi-AZ for high availability.

8. Common Use Cases



9. Example RDS Subnet Group

The subnet group should include private subnets in at least two Availability Zones.

Subnet	Availability Zone	Type
subnet-0a1b2c3d	us-east-1a	Private
subnet-0d4e5f6g	us-east-1b	Private

10. Best Practices

- ✓ Use private subnets.
- ✓ Restrict access with security groups.
- ✓ Enable encryption at rest.
- ✓ Use automated backups.
- ✓ Use Multi-AZ for production.
- ✓ Monitor database metrics.



Junior Engineer Tip

Use the RDS endpoint (not a public IP) in your application configuration, and keep your database credentials out of the code.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

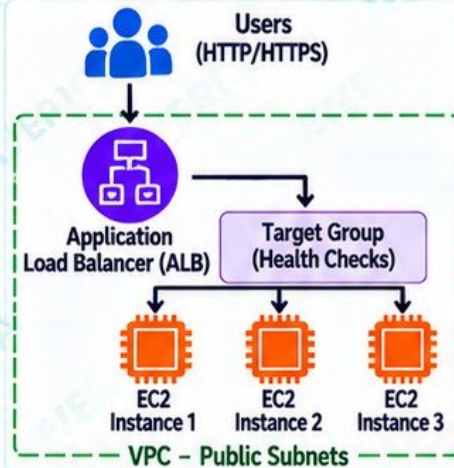


Distribute Traffic with an ALB

1. What is an Application Load Balancer?

- An Application Load Balancer (ALB) distributes incoming HTTP/HTTPS traffic across multiple EC2 instances.
- It improves availability, scalability and fault tolerance.
- Performs health checks and routes traffic only to healthy instances.
- Works at Layer 7 (Application layer) and can route based on paths, host names or other rules.

2. Architecture Overview



3. Create an ALB (Console)

- 1 Open the EC2 console.
- 2 Go to Load Balancers → Create load balancer.
- 3 Choose Application Load Balancer.
- 4 Select VPC and at least two public subnets (different AZs).
- 5 Configure a security group (allow HTTP/HTTPS).
- 6 Create a target group.
- 7 Register your EC2 instances.
- 8 Create and complete the load balancer.

4. Target Group and Health Checks

- 1 Create a target group (instance type).
- 2 Choose protocol (HTTP) and port (80).
- 3 Configure a health check path (e.g. / or /health).
- 4 Register multiple EC2 instances.
- 5 ALB routes traffic only to healthy instances.

 If an instance fails the health check, ALB stops sending traffic to it automatically.

5. Example Health Check Configuration

Setting	Value (Example)
Protocol	HTTP
Port	80
Path	/health
Healthy threshold	2
Unhealthy threshold	2
Timeout	5 seconds
Interval	30 seconds

6. Security Group Configuration

ALB Security Group (Public)

Type	Protocol	Port	Source	Description
HTTP	TCP	80	0.0.0.0/0	Allow web traffic
HTTPS	TCP	443	0.0.0.0/0	Allow secure traffic

EC2 Security Group (Instances)

Type	Protocol	Port	Source	Description
HTTP	TCP	80	ALB SG	Allow traffic from ALB only

✓ Do not allow 0.0.0.0/0 to the instances. Allow traffic only from the ALB security group.

7. Test the Load Balancer

- 1 Get the ALB DNS name from the console.
- 2 Open the DNS name in a browser.
- 3 Refresh multiple times to see traffic going to different instances.
- 4 Stop one instance and verify ALB routes traffic to healthy instances only.

Example (Check Targets with AWS CLI)

```
# Describe target health
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:
  region:123456789012:targetgroup/my-tg/123abc

# Get load balancer DNS name
aws elbv2 describe-load-balancers \
  --names my-alb \
  --query 'LoadBalancers[0].DNSName' \
  --output text
```

8. How ALB Routes Traffic

- ✓ Uses a listener (HTTP/HTTPS) to receive traffic.
- ✓ Forwards requests to a target group.
- ✓ Performs health checks on each instance.
- ✓ Routes only to healthy instances.
- ✓ Automatically stops sending traffic to unhealthy instances.
- ✓ Supports path-based and host-based routing rules.

9. Common Use Cases



Web applications



Microservices



APIs



E-commerce platforms



Content-heavy websites

10. Important Notes

- ! Use at least two subnets (different Availability Zones).
- ! Keep instances in private subnets if not directly exposed.
- ! Configure proper health checks.
- ! Use HTTPS in production.
- ! Combine with Auto Scaling for better scalability.



Junior Engineer Tip

Use an Application Load Balancer in front of multiple EC2 instances to improve availability and automatically handle instance failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

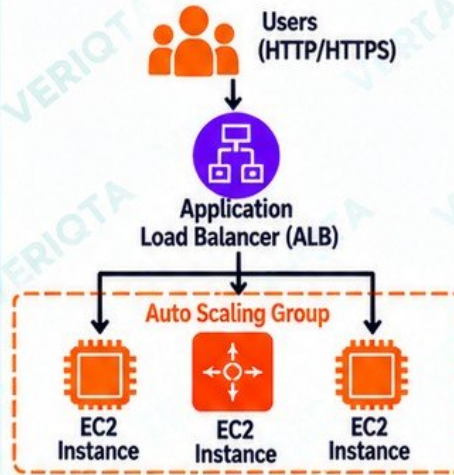


Scale EC2 Automatically

1. Why Use Auto Scaling?

- ✓ Automatically adjusts the number of EC2 instances based on demand.
- ✓ Improves availability and fault tolerance.
- ✓ Handles traffic spikes without manual intervention.
- ✓ Replaces unhealthy instances automatically.
- ✓ Helps optimize costs by scaling in when demand is low.

2. Architecture Overview



3. Create a Launch Template

- 1 Open the EC2 console.
- 2 Go to Launch Templates → Create launch template.
- 3 Provide a name and description.
- 4 Choose Amazon Machine Image (AMI).
- 5 Select an instance type.
- 6 Configure key pair, security groups and user data (optional).
- 7 Create the launch template.



A launch template defines the configuration for EC2 instances that will be launched by the Auto Scaling Group.

4. Create an Auto Scaling Group

- 1 Go to EC2 → Auto Scaling Groups.
- 2 Choose Create Auto Scaling group.
- 3 Select the launch template.
- 4 Choose the VPC and subnets (at least two Availability Zones).
- 5 Configure group size (minimum, desired and maximum).
- 6 Attach to a load balancer (optional but recommended).
- 7 Add scaling policies (e.g. based on CPU).
- 8 Create the Auto Scaling group.

5. Set Group Size

Minimum Capacity	Desired Capacity	Maximum Capacity
2	2	5
Minimum number of instances to maintain.	Initial number of instances at launch.	Maximum number of instances during scaling.



The Auto Scaling Group automatically launches or terminates instances to keep the capacity within this range.

6. Configure a Scaling Policy

- 1 Create a target tracking policy e.g. CPU utilization.
- 2 Set the target value (e.g. 70 percent).
- 3 Choose the metric (Average CPU Utilization).
- 4 Set cooldown period (e.g. 300 seconds).
- 5 Create the policy.

7. Example Scaling Policy (Target Tracking)



```
{
  "TargetTrackingConfiguration": {
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ASGAverageCPUUtilization"
    },
    "TargetValue": 70.0,
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 300
  }
}
```

8. Trigger and Observe Scaling

- 1 Generate traffic to increase CPU (e.g. using a load test).
- 2 Observe new instances being launched automatically.
- 3 Stop the traffic and watch the group scale in.
- 4 Terminate an instance manually and observe it being replaced.

9. Verify in the Console

EC2 Auto Scaling Group

Details	Activity	Monitoring
Desired capacity	2	
Minimum capacity	2	
Maximum capacity	5	
Instances	2 (InService)	
Availability Zones	us-east-1a, us-east-1b	
Load balancer	app/my-alb	
Launch template	veriqta-lt	

10. Common Use Cases



Handle traffic spikes



High availability applications



Web applications



Microservices



E-commerce platforms

11. Important Notes



- Use at least two subnets in different Availability Zones.
- Attach a load balancer for better traffic distribution.
- Choose appropriate scaling metrics (CPU, custom metrics).
- Set reasonable minimum, desired and maximum values.
- Use health checks to replace unhealthy instances.
- Monitor scaling activities and costs.



Junior Engineer Tip

Use a launch template and Auto Scaling Group to keep your application highly available, automatically handle traffic increases and replace failed instances.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



VERIQTA

Monitor Infrastructure with CloudWatch

1. What is Amazon CloudWatch?

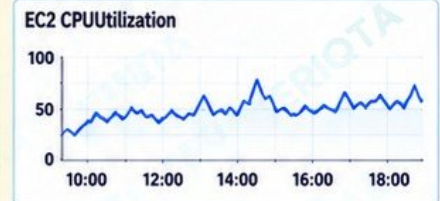
- Amazon CloudWatch is a monitoring and observability service for AWS resources and applications.
- Collects metrics, logs and events.
- Helps you detect problems, set alarms and take automated actions.
- Provides visibility into the health, performance and usage of your infrastructure.
- Used for operational monitoring and production troubleshooting.

2. How CloudWatch Works



3. View Metrics in CloudWatch

- Open the CloudWatch console.
- Go to Metrics → All metrics.
- Choose a service (e.g. EC2).
- Select a metric (e.g. CPUUtilization).
- Choose a time range and statistic (e.g. Average).
- View the graph and analyze trends.



4. Create a CloudWatch Alarm

- Go to CloudWatch → Alarms → Create alarm.
- Select a metric (e.g. EC2 CPUUtilization).
- Set the condition (e.g. CPU > 70% for 5 minutes).
- Choose an action (e.g. SNS notification or Auto Scaling action).
- Add a name and description.
- Create the alarm.



Alarms help you detect issues early and can trigger automated responses.

5. Work with Logs (CloudWatch Logs)

- Open CloudWatch → Logs → Log groups.
- Create a log group (or use an existing one).
- Send application or system logs to CloudWatch Logs.
- View and search logs using Log Insights.

View Logs using AWS CLI

```
# List log groups
aws logs describe-log-groups

# View log events
aws logs describe-log-streams \
  --log-group-name /aws/ec2/web-server

# Search logs using Log Insights
fields @timestamp, @message
| sort @timestamp desc
| limit 20
```

6. Important CloudWatch Metrics

Service	Common Metrics	Use Case
EC2	CPUUtilization NetworkIn/Out StatusCheckFailed	Monitor instance health and performance
RDS	CPUUtilization FreeStorageSpace DatabaseConnections	Monitor database performance and availability
ALB	RequestCount TargetResponseTime HTTPCode_Target_5XX	Monitor application traffic and healthy targets
S3	NumberOfRequests BucketSizeBytes	Monitor storage usage and requests



Choose metrics that are useful for your application and set meaningful thresholds based on real usage.

7. Identify Unhealthy Resources

- Use metrics to detect high CPU, memory or latency.
- Check status check failures on EC2 instances.
- Monitor RDS for high CPU, low storage or connection issues.
- Use ALB metrics to detect unhealthy targets.
- Review logs for errors and exceptions.
- Set up alarms to get notified before users are affected.

8. Example: Create an Alarm with AWS CLI

```
aws cloudwatch put-metric-alarm \
  --alarm-name High-CPU-Alarm \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 70 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:veriqta-alerts
```

9. Best Practices

- Monitor key metrics for all critical resources.
- Set alarms with meaningful thresholds.
- Use CloudWatch Logs for application logs.
- Centralize logs for easier troubleshooting.
- Use dashboards for quick visibility.
- Set up notifications via SNS.
- Automate responses using Auto Scaling or EventBridge.
- Regularly review and adjust your alarms.

10. Common Use Cases



Infrastructure monitoring



Alerting and notifications



Log analysis and troubleshooting



Performance optimization



Security monitoring



Automated remediation

11. Important Notes

- Monitor both metrics and logs.
- Set alarms for critical resources.
- Use appropriate thresholds.
- Review and investigate alarms.
- Keep logs for operational visibility.
- Combine with automation for faster response.



Junior Engineer Tip

Start with basic metrics and alarms, then expand to logs and dashboards as your application grows.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

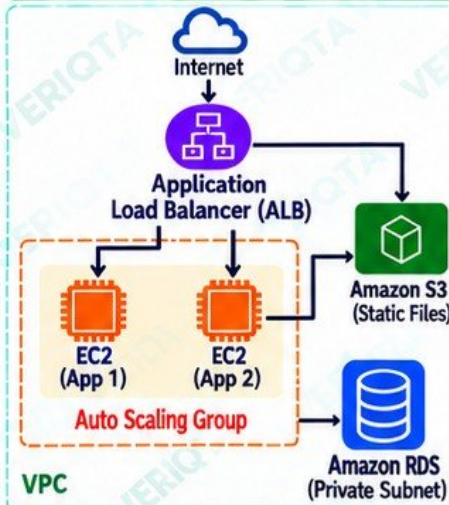


Troubleshoot a Broken AWS Application

1. Common Failure Areas

-  **Security Groups**
Blocked ports or incorrect rules.
-  **Network Connectivity**
Route table, subnet or NAT issues.
-  **Unhealthy Targets**
EC2 instances failing health checks.
-  **Services and Configuration**
Application or web server not running.
-  **IAM Permissions**
Missing or incorrect permissions.
-  **Database Connectivity**
RDS not accessible or wrong credentials.
-  **Load Balancer Issues**
Listeners, target groups or routing issues.

2. Example Architecture



3. Troubleshooting Approach

- 1 Confirm the symptoms (e.g. timeout, 4xx, 5xx, no response).
- 2 Identify the affected component (ALB, EC2, RDS, S3, etc).
- 3 Gather evidence (logs, metrics, status checks, error messages).
- 4 Form a hypothesis.
- 5 Test and validate the cause.
- 6 Apply the fix.
- 7 Verify the application is working.

4. Real-World Failure Scenarios

Scenario	Possible Cause	What to Check
Cannot access web application	Security group blocking port 80	EC2 SG inbound rules, ALB SG rules
ALB shows unhealthy targets	Application not running or wrong health check	Target group health, EC2 instance logs
EC2 cannot reach internet	Missing NAT gateway or route	Route table, subnet, NAT gateway
Application cannot access RDS	RDS security group or network issue	RDS SG, EC2 SG, subnet, route table
Access denied to S3	Missing IAM permissions	IAM role policy, CloudTrail logs
High error rate in application	Service misconfiguration or dependency failure	CloudWatch logs, service status, metrics

5. Useful AWS CLI Commands

```

# Check EC2 instance status
aws ec2 describe-instances \
  --query "Reservations[*].Instances[*].\
    [InstanceId,State,Name,PrivateIpAddress]" \
  --output table >-

# Check target group health
aws elbv2 describe-target-health \
  --target-group-arn <target-group-arn> \
  --output table >-

# Check security group rules
aws ec2 describe-security-groups \
  --group-ids <sg-id> \
  --output table >-

# Check CloudWatch logs (last 20 lines)
aws logs tail /aws/ec2/web-server \
  --since 20m >-

# Check RDS status
aws rds describe-db-instances \
  --query "DBInstances[*].[DBInstanceIdentifier,\
    DBInstanceStatus,Endpoint.Address]" \
  --output table >-
  
```

6. Where to Find Evidence

-  **CloudWatch Metrics**
Check CPU, memory, request count, error rates and alarms.
-  **CloudWatch Logs**
View application and system logs for errors and exceptions.
-  **ALB / Target Groups**
Check target health and request counts.
-  **EC2 Instance**
Check status, system logs and application logs.
-  **RDS**
Check status, connectivity and logs.
-  **VPC and Route Tables**
Verify routes, subnets and NAT gateway.
-  **IAM**
Review permissions and use CloudTrail for access denied errors.

7. Diagnose and Fix Example (Security Group Issue)

Problem	Check	Fix
Application not accessible (timeout).	✓ Check ALB security group inbound rules. ✓ Verify EC2 security group allows traffic from ALB. ✓ Confirm target group health	✓ Allow port 80 on ALB SG. ✓ Allow traffic from ALB SG to EC2 SG. ✓ Verify targets are healthy. ✓ Test the application again.

9. Troubleshooting Checklist

- ✓ Is the service running and healthy?
- ✓ Are security groups and network routes correct?
- ✓ Are the targets healthy?
- ✓ Are IAM permissions correct?
- ✓ Are there errors in application or system logs?
- ✓ Are dependent services (RDS, S3, etc) accessible?

10. Common Error Messages and Meaning

Error Message	Meaning
Connection timed out	Network, security group or route issue.
503 Service Unavailable	Unhealthy targets or application issue.
AccessDenied	IAM permission problem.
Target is unhealthy	Application not passing health checks.
No route to host	Routing or subnet configuration issue.
Could not connect to database	RDS security group or network issue.

8. Common Tools and Services


CloudWatch
(Metrics & Logs)


VPC
(Networking)


EC2
(Compute)


RDS
(Database)


ALB
(Load Balancer)


IAM
(Permissions)


CloudTrail
(API Activity)


S3
(Storage)

11. Important Notes

- ! Always check multiple layers (network, security, application, permissions).
- ! Use CloudWatch and logs for evidence.
- ! Make one change at a time and test.
- ! Document the root cause and fix.
- ! Set up monitoring and alarms to detect issues early.

Junior Engineer Tip

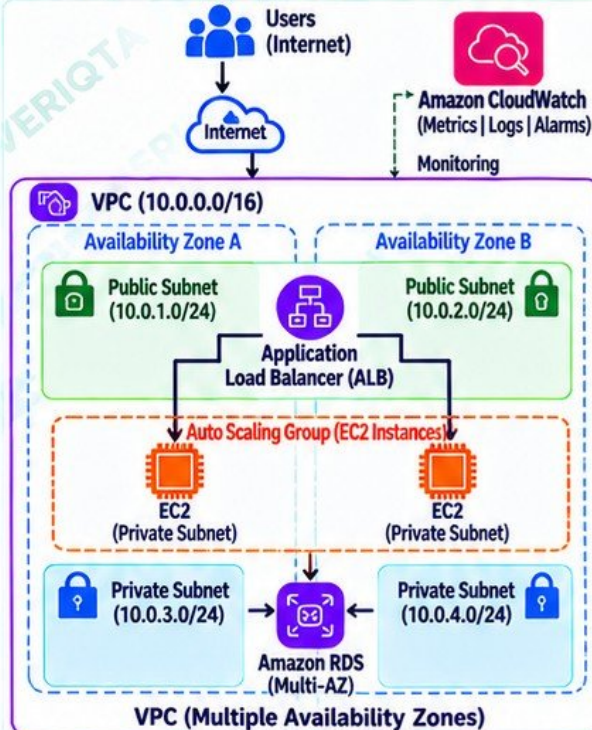
When troubleshooting, follow the evidence. Check network, security, service, logs and permissions step by step instead of guessing.

Build a Highly Available Architecture

1. Key Components and Purpose

-  **VPC**
Isolated network for your resources.
-  **Public Subnets**
Host internet-facing resources (such as ALB).
-  **Private Subnets**
Host application and database resources (no direct internet access).
-  **Application Load Balancer (ALB)**
Distributes traffic across multiple EC2 instances.
-  **Auto Scaling Group (ASG)**
Automatically adds or removes EC2 instances based on demand.
-  **Amazon RDS**
Managed database deployed in private subnets across multiple AZs.
-  **CloudWatch**
Monitoring, logs and alarms.
-  **IAM**
Controls access to AWS resources.

2. High Availability Architecture Overview



3. How It Works

- 1 Users access the application through the ALB.
- 2 The ALB distributes traffic to healthy EC2 instances across multiple AZs.
- 3 EC2 instances are in an Auto Scaling Group for high availability.
- 4 The application servers run in private subnets.
- 5 The database (RDS) is deployed in private subnets with Multi-AZ for resilience.
- 6 IAM controls access to AWS resources.
- 7 CloudWatch monitors metrics, logs and alarms.
- 8 If an instance or AZ fails, traffic is automatically routed to healthy resources in the other AZ.

4. Subnet Design (Example)

Subnet Type	AZ	CIDR	Purpose
Public Subnet	AZ A	10.0.1.0/24	ALB, NAT Gateway (bastion if needed)
Public Subnet	AZ B	10.0.2.0/24	ALB, NAT Gateway (bastion if needed)
Private Subnet	AZ A	10.0.3.0/24	EC2 instances (application tier)
Private Subnet	AZ B	10.0.4.0/24	EC2 instances (application tier)
Private Subnet	AZ A	10.0.5.0/24	RDS (database tier)
Private Subnet	AZ B	10.0.6.0/24	RDS (database tier)

5. Security Groups (Example)

Resource	Inbound Rules	Purpose
ALB SG	HTTP/HTTPS from 0.0.0.0/0	Allow web traffic from the internet.
EC2 SG	HTTP from ALB SG SSH from your IP (optional)	Allow traffic from ALB to EC2 instances.
RDS SG	MySQL/PostgreSQL from EC2 SG	Allow database access only from application instances.

6. Auto Scaling Configuration

-  **Launch Template**
 - Choose Amazon Machine Image (AMI)
 - Instance type (e.g. t3.micro)
 - Key pair (if needed)
 - Security group
 - User data (install application)
-  **Auto Scaling Group**
 - Minimum capacity (e.g. 2)
 - Desired capacity (e.g. 2)
 - Maximum capacity (e.g. 6)
 - Select multiple AZs
 - Attach to the ALB target group

7. RDS Configuration (Multi-AZ)

-   Choose database engine (e.g. MySQL, PostgreSQL).
-  Deploy in private subnets (different AZs).
-  Enable Multi-AZ for high availability.
-  Configure security group (allow only from EC2).
-  Set backup and maintenance windows.
-  Use encrypted storage.

8. Monitoring and Alerts (CloudWatch)

-   Monitor EC2 (CPU, status check).
-  Monitor ALB (request count, target health).
-  Monitor RDS (CPU, connections, free storage).
-  Create alarms (e.g. high CPU, unhealthy targets).
-  Send notifications via SNS (email, SMS, etc.).
-  Use logs for troubleshooting (EC2, ALB, RDS).

9. High Availability Benefits

-  Survives instance failures automatically.
-  Distributes traffic across multiple AZs.
-  Keeps the database private and resilient.
-  Scales based on demand.
-  Provides monitoring and alerting.
-  Improves security and reliability.

10. Common Use Cases

-  Web applications
-  Business applications
-  E-commerce platforms
-  Microservices
-  Enterprise applications

11. Important Notes

-  Always use at least two Availability Zones.
-  Keep databases in private subnets.
-  Use security groups with least privilege.
-  Enable monitoring and alarms.
-  Plan for backups and disaster recovery.
-  Test the architecture by simulating failures.

Junior Engineer Tip

A highly available architecture uses multiple Availability Zones, proper network design, load balancing, Auto Scaling, and monitoring to keep your application running even when failures occur.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

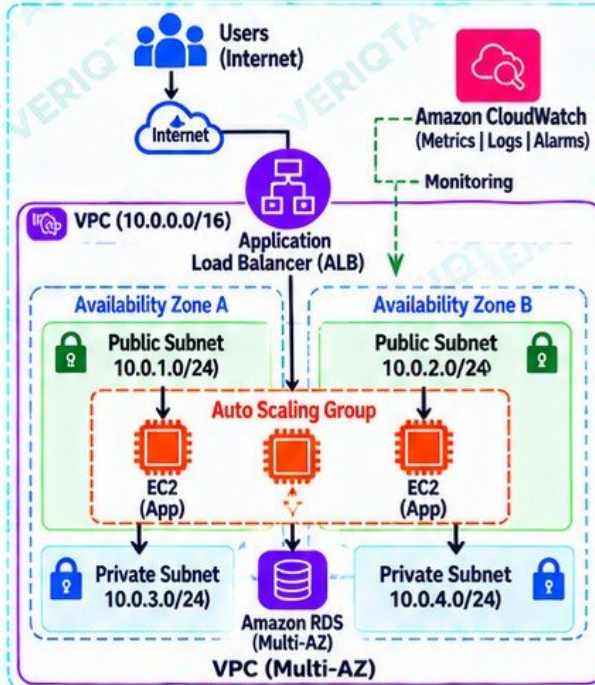


Final Infrastructure Challenge

1. Challenge Goal

- ✓ Build a complete and secure AWS environment with fewer instructions.
- ✓ Validate security, connectivity and availability.
- ✓ Test failure recovery.
- ✓ Document the architecture and decisions.
- ✓ Clean up all resources at the end.

2. Example Architecture (Build This)



3. Key Requirements

- 1 Create a VPC with public and private subnets across multiple AZs.
- 2 Launch an ALB and an Auto Scaling Group with EC2 instances.
- 3 Deploy a database using RDS in private subnets (Multi-AZ).
- 4 Configure security groups with least privilege.
- 5 Set up monitoring with CloudWatch (metrics, logs and alarms).
- 6 Validate connectivity and application availability.
- 7 Test failure recovery (e.g. terminate an instance and observe replacement).
- 8 Document the architecture and clean up all resources.

4. Validation Checklist

- ✓ Application is accessible through ALB.
- ✓ EC2 instances are in an Auto Scaling Group.
- ✓ Database is in private subnets and not publicly accessible.
- ✓ Security groups follow least privilege.
- ✓ Application works across multiple AZs.
- ✓ Monitoring and alarms are configured.
- ✓ Instance replacement works after failure.
- ✓ All resources are documented with architecture diagram.
- ✓ All resources are cleaned up at the end.

5. Failure Recovery Test

- 1 Manually terminate one EC2 instance in the Auto Scaling Group.
- 2 Verify a new instance is automatically launched.
- 3 Check ALB target group shows healthy instances.
- 4 Confirm the application remains available.
- 5 Review CloudWatch logs and metrics during the event.

6. Documentation

- Architecture diagram (with AWS icons)
- List of all resources created
- Security design and decisions
- Testing results and observations
- Lessons learned and improvements

7. Security Considerations

- Use least privilege IAM roles and policies
- Restrict access with security groups
- Keep databases in private subnets
- Use IAM roles instead of access keys
- Enable encryption (in transit and at rest)

8. Useful AWS Services

- EC2 (Compute)
- ALB (Load Balancer)
- Auto Scaling (Scaling)
- RDS (Database)
- CloudWatch (Monitoring)
- IAM (Access Control)
- VPC (Networking)

9. Common Checks and Commands

```
# Check EC2 instances in the Auto Scaling Group
aws autoscaling describe-auto-scaling-groups --query "AutoScalingGroups[*].Instances[*].[InstanceId,LifecycleState]"

# Check ALB target health
aws elbv2 describe-target-health --target-group-arn <target-group-arn>

# Check RDS instance status
aws rds describe-db-instances --query "DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus]"

# View CloudWatch EC2 CPU metric
aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization \
--dimensions Name=InstanceId,Value=<Instance-id> --statistics Average --period 300 --start-time <start> --end-time <end>
```

10. Final Deliverables

- ✓ Working AWS environment.
- ✓ Validated security, connectivity and availability.
- ✓ Failure recovery test results.
- ✓ Architecture diagram.
- ✓ Summary of key decisions.
- ✓ All resources cleaned up.

Junior Engineer Tip

Think like a real engineer. Build, test, break, recover, document and clean up. This is how you gain confidence in AWS.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

